

MODUL PRAKTIKUM STRUKTUR DATA

Modul 7



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

PERTEMUAN 7

POHON PENCARIAN BINER (BINARY SEARCH TREE / BST)

1. DESKRIPSI MODUL

Modul ini membahas Binary Search Tree (BST) yang merupakan pengembangan dari Binary Tree dengan aturan khusus dalam penyimpanan data. BST dirancang untuk mempercepat proses pencarian, penambahan, dan penghapusan data dibandingkan struktur data linear.

Pada BST, setiap node memiliki maksimal dua child, yaitu left child dan right child. Namun berbeda dengan Binary Tree biasa, BST memiliki aturan bahwa nilai pada subtree kiri harus lebih kecil dari node induk, sedangkan nilai pada subtree kanan harus lebih besar dari node induk.

Karena memiliki struktur yang teratur, BST menjadi salah satu struktur data yang banyak digunakan dalam sistem pencarian, database, sistem file, dan berbagai aplikasi yang membutuhkan akses data secara cepat.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Binary Search Tree.
- Memahami aturan penyusunan node pada BST.
- Mengimplementasikan operasi insert pada BST.
- Mengimplementasikan operasi search pada BST.
- Memahami konsep delete pada BST.
- Menganalisis efisiensi BST dibandingkan struktur data linear.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep BST.
- Membuat BST sederhana menggunakan Python.
- Mengimplementasikan operasi insert.
- Mengimplementasikan operasi search.
- Memahami proses delete node.

- Menganalisis keunggulan BST dalam pencarian data.

4. DASAR TEORI

4.1 Pengertian Binary Search Tree

Binary Search Tree (BST) adalah Binary Tree yang memiliki aturan khusus:

Aturan BST = Untuk setiap node: Nilai kiri < Root < Nilai kanan

Contoh BST:

```

      50
     /  \
    30   70
   /\   /\
  20 40 60 80

```

4.2 Mengapa BST Digunakan?

Pada array biasa: [10,20,30,40,50,60]. Pencarian sering dilakukan satu per satu.

Pada BST:

```

      40
     /  \
    20   60
   /\   /\
  10 30 50 70

```

Pencarian dapat dilakukan lebih cepat karena sebagian data dapat langsung diabaikan.

4.3 Struktur Node BST

Setiap node memiliki Data, Left Child, dan Right Child.

Contoh: [Data | Left | Right]

4.4 Operasi Dasar BST

- **Insert** = Menambahkan node sesuai aturan BST.
- **Search** = Mencari node berdasarkan nilai tertentu.
- **Delete** = Menghapus node dari BST.
- **Traversal** = Menampilkan seluruh isi BST.

4.5 Keunggulan BST

- Data tersusun secara terurut.
- Pencarian lebih cepat.
- Insert lebih efisien.
- Cocok untuk data dinamis.

4.6 Kompleksitas BST

Operasi	Kompleksitas Rata-rata
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$

Jika BST tidak seimbang:

10

\

20

\

30

\

40

Kompleksitas dapat menjadi: $O(n)$

4.7 Perbedaan Binary Tree dan BST

Binary Tree	BST
Tidak memiliki aturan nilai	Memiliki aturan nilai
Traversal tidak selalu terurut	Inorder menghasilkan data terurut
Pencarian lebih lambat	Pencarian lebih cepat

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Node BST

```
class Node:

    def __init__(self, key):

        self.key = key

        self.left = None

        self.right = None
```

```
root = Node(50)

print(root.key)
```

Analisis = Membuat node pertama sebagai root BST.

Praktikum 2 – Insert Manual

```
root = Node(50)

root.left = Node(30)

root.right = Node(70)
```

Struktur:

```
    50
   / \
  30  70
```

Praktikum 3 – Fungsi Insert BST

```
def insert(root, key):

    if root is None:

        return Node(key)

    if key < root.key:

        root.left = insert(root.left, key)

    else:

        root.right = insert(root.right, key)

    return root
```

Praktikum 4 – Menambahkan Data ke BST

```
root = None

root = insert(root, 50)

root = insert(root, 30)

root = insert(root, 70)

root = insert(root, 20)

root = insert(root, 40)
```

Hasil:

```
    50
  /  \
 30   70
 / \
20 40
```

Praktikum 5 – Traversal Inorder

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

inorder(root)
```

Output: 20 30 40 50 70

Analisis = Traversal inorder pada BST menghasilkan data yang sudah terurut.

Praktikum 6 – Search BST

```
def search(root, key):
    if root is None:
        return False

    if root.key == key:
        return True

    if key < root.key:
        return search(root.left, key)
```

```
return search(root.right, key)
```

Praktikum 7 – Mencari Data

```
print(search(root, 40))
```

```
print(search(root, 100))
```

Output:

True

False

Praktikum 8 – Visualisasi Jalur Pencarian

Cari nilai:

40

Pada BST:

```
      50
     /  \
    30   70
   /  \
  20  40
```

Langkah:

50 → 30 → 40

Data ditemukan hanya dalam beberapa langkah.

Praktikum 9 – Konsep Delete Node

Kasus:

Hapus:

20

Sebelum:

```

      50
     /  \
    30   70
   /  \
  20  40

```

Sesudah:

```

      50
     /  \
    30   70
     \
      40

```

Praktikum 10 – Perbandingan Linear Search dan BST

Data:

10.000 data

Bandingkan:

- Linear Search $\rightarrow O(n)$
- BST $\rightarrow O(\log n)$

Diskusikan perbedaannya.

7. STUDI KASUS

Studi Kasus 1

Pencarian data mahasiswa berdasarkan NIM.

Studi Kasus 2

Pencarian produk pada toko online.

Studi Kasus 3

Pencarian kontak pada sistem telepon.

Studi Kasus 4

Pencarian data pasien rumah sakit.

Studi Kasus 5

Pencarian file dalam sistem komputer.

8. TUGAS PRAKTIKUM

Tugas 1

Buat BST dengan data:

50, 30, 70, 20, 40, 60, 80

Tugas 2

Gambarkan struktur BST yang terbentuk.

Tugas 3

Lakukan traversal inorder.

Tugas 4

Lakukan traversal preorder.

Tugas 5

Lakukan traversal postorder.

Tugas 6

Cari nilai: 40

Tugas 7

Cari nilai: 90

Tugas 8

Jelaskan langkah pencarian nilai 60.

Tugas 9

Bandingkan BST dan Binary Tree biasa.

Tugas 10

Bandingkan BST dan Linear Search.

9. DISKUSI

1. Apa yang dimaksud BST?
2. Mengapa BST lebih cepat daripada pencarian linear?
3. Apa aturan utama BST?
4. Apa fungsi traversal inorder pada BST?
5. Mengapa BST dapat menghasilkan data terurut?
6. Apa kelemahan BST yang tidak seimbang?
7. Kapan BST lebih cocok digunakan dibanding array?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan

MODUL PRAKTIKUM STRUKTUR DATA

Modul 8



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 8

HEAP DAN ANTRIAN PRIORITAS (PRIORITY QUEUE)

1. DESKRIPSI MODUL

Modul ini membahas struktur data Heap dan Priority Queue (Antrian Prioritas) yang digunakan untuk mengelola data berdasarkan tingkat prioritas tertentu. Berbeda dengan Queue biasa yang menggunakan prinsip FIFO (First In First Out), Priority Queue akan memproses data berdasarkan prioritas tertinggi atau terendah terlebih dahulu.

Heap merupakan struktur data berbentuk Binary Tree khusus yang memenuhi aturan tertentu sehingga memungkinkan operasi pencarian nilai minimum atau maksimum dilakukan secara efisien.

Heap banyak digunakan dalam sistem operasi, penjadwalan proses CPU, sistem rumah sakit, navigasi GPS, simulasi event, dan berbagai aplikasi yang memerlukan pengelolaan prioritas.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Heap.
- Memahami perbedaan Min Heap dan Max Heap.
- Mengimplementasikan operasi dasar Heap.
- Memahami konsep Priority Queue.
- Mengimplementasikan Priority Queue menggunakan Python.
- Menganalisis penggunaan Heap dalam sistem prioritas.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Heap.
- Membedakan Min Heap dan Max Heap.
- Mengimplementasikan operasi insert dan delete pada Heap.
- Menggunakan modul heapq pada Python.
- Mengimplementasikan Priority Queue sederhana.

- Menganalisis kasus nyata penggunaan prioritas.

4. DASAR TEORI

4.1 Pengertian Heap

Heap adalah struktur data berbentuk Binary Tree yang memenuhi aturan tertentu.

Heap digunakan untuk:

- Menemukan nilai minimum dengan cepat.
- Menemukan nilai maksimum dengan cepat.
- Mengelola prioritas data.

4.2 Karakteristik Heap

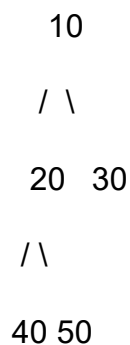
Heap merupakan:

- Complete Binary Tree
- Memiliki hubungan parent-child
- Mengikuti aturan heap property

Complete Binary Tree

Semua level terisi penuh kecuali level terakhir.

Contoh:



4.3 Min Heap

Pada Min Heap: $\text{Parent} \leq \text{Child}$. Nilai terkecil selalu berada di root.

Contoh:

```

      10
  
```

/ \

20 30

/ \

40 50

Nilai minimum:

10

4.4 Max Heap

Pada Max Heap: Parent $\geq$ Child. Nilai terbesar selalu berada di root.

Contoh:

90

/ \

70 50

/ \

30 20

Nilai maksimum:

90

4.5 Heap Property

Heap hanya menjamin hubungan antara parent dan child. Heap tidak menjamin seluruh data terurut.

Contoh valid:

10

/ \

20 15

/ \

50 40

Karena:

$10 < 20$

$10 < 15$

$20 < 50$

$20 < 40$

4.6 Priority Queue

Priority Queue adalah antrian yang memproses data berdasarkan prioritas. Bukan berdasarkan urutan kedatangan.

Queue Biasa

Andi

Budi

Citra

Diproses: Andi → Budi → Citra

Priority Queue

Andi (Prioritas 2)

Budi (Prioritas 1)

Citra (Prioritas 3)

Diproses: Budi → Andi → Citra

4.7 Keunggulan Heap

- Insert cepat
- Delete cepat
- Mendukung Priority Queue
- Efisien untuk data besar

4.8 Kompleksitas Heap

Operasi	Kompleksitas
Insert	$O(\log n)$
Delete	$O(\log n)$
Peek	$O(1)$
Build Heap	$O(n)$

4.9 Perbedaan BST dan Heap

BST	Heap
Untuk pencarian	Untuk prioritas
Data relatif terurut	Tidak sepenuhnya terurut
Search cepat	Priority cepat
Root belum tentu terbesar/terkecil	Root selalu min/max

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Min Heap

```
import heapq
```

```
heap = []
```

```
heapq.heappush(heap, 20)
```

```
heapq.heappush(heap, 10)
```

```
heapq.heappush(heap, 30)
```

```
print(heap)
```

Output = [10, 20, 30]

Analisis = Nilai terkecil otomatis berada di posisi root.

Praktikum 2 – Menambahkan Data ke Heap

```
import heapq
```

```
heap = [10,20,30]
```

```
heapq.heappush(heap,5)
```

```
print(heap)
```

Output = [5,10,30,20]

Analisis = Heap akan menyesuaikan posisi data agar aturan Min Heap tetap terpenuhi.

Praktikum 3 – Mengambil Nilai Minimum

```
import heapq
```

```
heap = [5,10,20,30]
```

```
nilai = heapq.heappop(heap)
```

```
print(nilai)
```

```
print(heap)
```

Output = 5

[10,30,20]

Praktikum 4 – Melihat Nilai Prioritas Tertinggi

```
print(heap[0])
```

Output = 10

Praktikum 5 – Heapify

```
import heapq

data = [50,20,10,40,30]

heapq.heapify(data)

print(data)
```

Analisis = Mengubah list biasa menjadi heap.

Praktikum 6 – Simulasi Prioritas Pasien Rumah Sakit

```
import heapq

antrian = []

heapq.heappush(antrian, (2, "Pasien A"))

heapq.heappush(antrian, (1, "Pasien Darurat"))

heapq.heappush(antrian, (3, "Pasien B"))

print(antrian)
```

Praktikum 7 – Melayani Pasien Prioritas

```
pasien = heapq.heappop(antrian)

print("Dilayani:", pasien)
```

Output = Dilayani: (1, 'Pasien Darurat')

Praktikum 8 – Priority Queue Tugas Kuliah

```
import heapq

tugas = []

heapq.heappush(tugas, (1, "Tugas Besok"))

heapq.heappush(tugas, (3, "Tugas Minggu Depan"))

heapq.heappush(tugas, (2, "Tugas 3 Hari Lagi"))

print(tugas)
```

Praktikum 9 – Simulasi Penjadwalan CPU

```
import heapq

proses = []

heapq.heappush(proses, (1, "Process A"))
heapq.heappush(proses, (5, "Process B"))
heapq.heappush(proses, (2, "Process C"))

while proses:
    print(heapq.heappop(proses))
```

Praktikum 10 – Perbandingan Queue dan Priority Queue

Buat simulasi:

Queue = $A \rightarrow B \rightarrow C$

Priority Queue = $A(2) \rightarrow B(1) \rightarrow C(3)$

Bandingkan hasil pemrosesan.

7. STUDI KASUS

Studi Kasus 1

Antrian pasien rumah sakit.

Studi Kasus 2

Penjadwalan CPU.

Studi Kasus 3

Sistem tiket prioritas bandara.

Studi Kasus 4

Manajemen tugas mahasiswa.

Studi Kasus 5

Navigasi GPS (jalur terpendek).

Studi Kasus 6

Sistem panggilan darurat 112.

8. TUGAS PRAKTIKUM

Tugas 1

Buat Min Heap dengan data: 30, 10, 50, 20, 40

Tugas 2

Tambahkan nilai: 5

Tugas 3

Hapus nilai minimum.

Tugas 4

Tampilkan isi heap setelah operasi.

Tugas 5

Buat simulasi Priority Queue pasien rumah sakit.

Tugas 6

Buat simulasi Priority Queue tugas kuliah.

Tugas 7

Bandingkan Queue dan Priority Queue.

Tugas 8

Jelaskan perbedaan Min Heap dan Max Heap.

Tugas 9

Analisis kompleksitas operasi Heap.

Tugas 10

Jelaskan mengapa Heap cocok digunakan untuk Priority Queue.

9. DISKUSI

1. Apa yang dimaksud Heap?
2. Apa perbedaan Min Heap dan Max Heap?
3. Mengapa root selalu berisi nilai minimum atau maksimum?
4. Apa yang dimaksud Priority Queue?
5. Mengapa Priority Queue berbeda dengan Queue biasa?
6. Apa keuntungan menggunakan Heap?
7. Kapan Heap digunakan dalam dunia nyata?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan

MODUL PRAKTIKUM STRUKTUR DATA

Modul 9



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 9

TABEL HASH (HASH TABLE)

1. DESKRIPSI MODUL

Modul ini membahas Hash Table, yaitu struktur data yang digunakan untuk menyimpan dan mengambil data dengan sangat cepat menggunakan teknik hashing. Berbeda dengan struktur data seperti array, linked list, atau tree yang membutuhkan proses pencarian bertahap, Hash Table memungkinkan data diakses langsung melalui sebuah kunci (key).

Hash Table merupakan salah satu struktur data yang paling banyak digunakan dalam pemrograman modern karena mampu memberikan performa pencarian, penambahan, dan penghapusan data yang sangat cepat. Dalam Python, implementasi Hash Table dapat ditemukan pada struktur data Dictionary.

Pada praktikum ini mahasiswa akan mempelajari konsep hashing, fungsi hash, collision (tabrakan data), teknik penanganan collision, serta implementasi Hash Table menggunakan Python.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Hash Table.
- Memahami proses hashing.
- Memahami fungsi hash.
- Menjelaskan collision dan cara mengatasinya.
- Mengimplementasikan Dictionary Python.
- Menganalisis efisiensi Hash Table dibandingkan struktur data lain.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Hash Table.
- Menjelaskan fungsi hash.
- Mengidentifikasi collision pada Hash Table.
- Mengimplementasikan Dictionary sebagai Hash Table.

- Menggunakan Hash Table untuk pencarian data.
- Menganalisis efisiensi akses data menggunakan Hash Table.

4. DASAR TEORI

4.1 Pengertian Hash Table

Hash Table adalah struktur data yang menyimpan data dalam pasangan: Key → Value

Contoh:

NIM → Nama

230101 → Andi

230102 → Budi

230103 → Citra

4.2 Mengapa Menggunakan Hash Table?

Pada list:

data = ["Andi", "Budi", "Citra"]

Untuk mencari: "Citra". Program harus memeriksa data satu per satu.

Pada Hash Table:

```

mahasiswa = {
    "230103": "Citra"
}

```

Data dapat diakses langsung menggunakan key.

4.3 Konsep Hashing

Hashing adalah proses mengubah key menjadi indeks penyimpanan menggunakan fungsi hash.

Ilustrasi:

Key



Hash Function



Index



Data Disimpan

4.4 Fungsi Hash

Fungsi hash digunakan untuk menentukan lokasi penyimpanan data.

Contoh sederhana:

$$\text{Hash}(\text{key}) = \text{key} \bmod 10$$

$$\text{Misal: } 25 \bmod 10 = 5$$

Maka data disimpan pada indeks 5.

4.5 Collision (Tabrakan)

Collision terjadi ketika dua key menghasilkan indeks yang sama.

Contoh:

$$15 \bmod 10 = 5$$

$$25 \bmod 10 = 5$$

Kedua data ingin masuk ke indeks yang sama.

4.6 Penanganan Collision

Separate Chaining

Setiap indeks menyimpan daftar data.

Contoh:

Index 5

↓

15 → 25

Linear Probing

Jika posisi penuh, cari posisi kosong berikutnya.

Contoh:

Index 5 = penuh

Simpan di Index 6

4.7 Dictionary pada Python

Dictionary merupakan implementasi Hash Table di Python.

Contoh:

```
mahasiswa = {  
    "230101": "Andi",  
    "230102": "Budi"  
}
```

4.8 Operasi Dasar Hash Table

- **Insert** = Menambahkan pasangan key-value.
- **Search** = Mencari data berdasarkan key.
- **Update** = Mengubah value berdasarkan key.
- **Delete** = Menghapus data berdasarkan key.

4.9 Kompleksitas Hash Table

Operasi	Kompleksitas
Insert	$O(1)$
Search	$O(1)$
Update	$O(1)$
Delete	$O(1)$

Dalam kondisi ideal, Hash Table sangat cepat.

4.10 Perbandingan Struktur Data

Struktur	Search
Array	$O(n)$
Linked List	$O(n)$
BST	$O(\log n)$
Hash Table	$O(1)$

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Dictionary

```
mahasiswa = {  
    "230101": "Andi",  
    "230102": "Budi"  
}  
  
print(mahasiswa)
```

Analisis = Dictionary menyimpan data dalam bentuk key-value.

Praktikum 2 – Mengakses Data

```
print(mahasiswa["230101"])
```

Output = Andi

Praktikum 3 – Menambahkan Data

```
mahasiswa["230103"] = "Citra"
```

```
print(mahasiswa)
```

Praktikum 4 – Mengubah Data

```
mahasiswa["230102"] = "Bambang"
```

```
print(mahasiswa)
```

Praktikum 5 – Menghapus Data

```
del mahasiswa["230101"]
```

```
print(mahasiswa)
```

Praktikum 6 – Mengecek Key

```
if "230102" in mahasiswa:
```

```
    print("Data ditemukan")
```

Praktikum 7 – Menampilkan Semua Data

```
for nim, nama in mahasiswa.items():
```

```
    print(nim, nama)
```

Praktikum 8 – Simulasi Data Mahasiswa

```
mahasiswa = {
```

```
    "230101": "Andi",
```

```
    "230102": "Budi",
```

```
    "230103": "Citra"
```

```
}
```

```
cari = "230103"
```

```
print(mahasiswa[cari])
```

Praktikum 9 – Simulasi Data Produk

```
produk = {  
    "P001": "Laptop",  
    "P002": "Mouse",  
    "P003": "Keyboard"  
}
```

```
print(produk["P002"])
```

Praktikum 10 – Simulasi Buku Perpustakaan

```
buku = {  
    "BK001": "Struktur Data",  
    "BK002": "Data Mining",  
    "BK003": "Machine Learning"  
}
```

```
print(buku["BK001"])
```

Praktikum 11 – Fungsi Hash Sederhana

```
def hash_function(key):  
    return key % 10  
  
print(hash_function(25))
```

Output = 5

Praktikum 12 – Simulasi Collision

```
print(15 % 10)
```

```
print(25 % 10)
```

Output:

5

5

Diskusikan mengapa collision terjadi.

7. STUDI KASUS

Studi Kasus 1

Pencarian data mahasiswa berdasarkan NIM.

Studi Kasus 2

Pencarian produk toko online.

Studi Kasus 3

Sistem login pengguna.

Studi Kasus 4

Daftar buku perpustakaan.

Studi Kasus 5

Kontak telepon digital.

Studi Kasus 6

Pencarian data pelanggan.

8. TUGAS PRAKTIKUM

Tugas 1

Buat Dictionary berisi 5 mahasiswa.

Tugas 2

Tambahkan data mahasiswa baru.

Tugas 3

Ubah salah satu data mahasiswa.

Tugas 4

Hapus salah satu mahasiswa.

Tugas 5

Cari mahasiswa berdasarkan NIM.

Tugas 6

Buat Dictionary produk toko.

Tugas 7

Buat fungsi hash sederhana menggunakan operasi modulo.

Tugas 8

Tunjukkan contoh collision.

Tugas 9

Bandingkan Hash Table dan BST.

Tugas 10

Analisis kelebihan Hash Table dalam pencarian data.

9. DISKUSI

1. Apa yang dimaksud Hash Table?
2. Apa fungsi hashing?
3. Mengapa Hash Table sangat cepat?
4. Apa yang dimaksud collision?
5. Bagaimana collision dapat terjadi?
6. Apa fungsi key dalam Dictionary?
7. Mengapa Dictionary Python disebut implementasi Hash Table?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan

MODUL PRAKTIKUM STRUKTUR DATA

Modul 10



Disusun Oleh:

Agus Nugraha, ST, M.Kom | Fitri Fatimah | Kanaya Dzikra

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS DAN TEKNOLOGI**

PERTEMUAN 10

GRAPH DAN PENELUSURAN GRAPH (BFS & DFS)

1. DESKRIPSI MODUL

Modul ini membahas struktur data non-linear Graph, yaitu struktur data yang digunakan untuk merepresentasikan hubungan antar objek atau entitas. Berbeda dengan Tree yang memiliki hubungan hierarki, Graph memiliki hubungan yang lebih fleksibel sehingga dapat digunakan untuk memodelkan berbagai sistem dunia nyata seperti jaringan komputer, media sosial, peta jalan, sistem transportasi, dan hubungan antar pengguna.

Pada praktikum ini mahasiswa akan mempelajari komponen utama Graph, representasi Graph menggunakan Adjacency List dan Adjacency Matrix, serta teknik traversal Graph menggunakan Breadth First Search (BFS) dan Depth First Search (DFS).

Pemahaman Graph sangat penting karena banyak digunakan dalam pengembangan sistem modern, kecerdasan buatan, analisis jaringan, dan optimasi rute.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

- Memahami konsep Graph.
- Memahami komponen node dan edge.
- Membedakan Graph dan Tree.
- Mengimplementasikan representasi Graph.
- Mengimplementasikan algoritma BFS.
- Mengimplementasikan algoritma DFS.
- Menganalisis traversal Graph pada berbagai kasus.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

- Menjelaskan konsep Graph.
- Mengidentifikasi node dan edge.
- Membuat representasi Graph menggunakan Python.

- Mengimplementasikan BFS.
- Mengimplementasikan DFS.
- Menganalisis jalur dan hubungan antar node.

4. DASAR TEORI

4.1 Pengertian Graph

Graph adalah struktur data non-linear yang terdiri dari:

- Vertex (Node) = merepresentasikan objek.
- Edge (Sisi) = merepresentasikan hubungan antar objek.

Contoh Graph

A ----- B

| |

| |

C ----- D

4.2 Komponen Graph

Vertex (Node)

Node adalah titik yang merepresentasikan suatu objek.

Contoh: A, B, C, D

Edge

Edge adalah hubungan antar node.

Contoh: A --- B. Artinya node A terhubung dengan node B.

4.3 Jenis-Jenis Graph

Undirected Graph

Hubungan dua arah.

Contoh: A ----- B

A terhubung ke B dan B terhubung ke A.

Directed Graph

Hubungan satu arah.

Contoh: A -----> B

A menuju B tetapi tidak selalu sebaliknya.

Weighted Graph

Graph yang memiliki bobot.

Contoh: A --5-- B

Angka 5 menunjukkan jarak atau biaya.

4.4 Graph dan Tree

Tree	Graph
Memiliki root	Tidak harus memiliki root
Tidak memiliki siklus	Bisa memiliki siklus
Hierarkis	Fleksibel
Jalur tunggal	Banyak jalur

4.5 Representasi Graph

Adjacency List

Representasi menggunakan daftar tetangga.

Contoh:

A → B, C

B → A, D

C → A, D

D → B, C

Dalam Python:

```
graph = {  
    "A": ["B", "C"],
```

```

    "B": ["A", "D"],

    "C": ["A", "D"],

    "D": ["B", "C"]

}

```

Adjacency Matrix

Representasi menggunakan matriks.

Contoh:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

4.6 Traversal Graph

Traversal adalah proses mengunjungi seluruh node pada Graph.

Dua algoritma traversal utama:

- BFS
- DFS

4.7 Breadth First Search (BFS)

BFS mengunjungi node berdasarkan level terdekat terlebih dahulu.

BFS menggunakan Queue dengan prinsip FIFO.

Contoh:

```

    A
  / \
 B   C
 / \

```

D E

Urutan BFS:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

4.8 Depth First Search (DFS)

DFS menjelajahi node sedalam mungkin sebelum kembali.

DFS menggunakan Stack dengan prinsip: LIFO

Contoh:

A

/ \

B C

/ \

D E

Urutan DFS:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

4.9 Kompleksitas BFS dan DFS

Algoritma	Kompleksitas
BFS	$O(V + E)$
DFS	$O(V + E)$

Keterangan:

- V = jumlah vertex
- E = jumlah edge

4.10 Penerapan Graph

Graph digunakan pada:

- Google Maps
- Media sosial
- Jaringan komputer
- Sistem transportasi
- Sistem rekomendasi
- Analisis hubungan data

5. ALAT DAN PERANGKAT

- Laptop / Komputer
- Python
- Google Colab
- Browser
- Modul Praktikum

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Graph Sederhana

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A"],
    "D": ["B"]
}
```

```
print(graph)
```

Praktikum 2 – Menampilkan Tetangga Node

```
print(graph["A"])
```

Output: ['B', 'C']

Praktikum 3 – Menambahkan Edge

```
graph["C"].append("D")

print(graph)
```

Praktikum 4 – Traversal BFS

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": [],
    "D": [],
    "E": []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            for neighbor in graph[node]:
                queue.append(neighbor)

bfs(graph, "A")
```

Output: A B C D E

Praktikum 5 – Analisis BFS

Diskusikan:

Mengapa urutan BFS: A B C D E

Bukan: A B D E C

Praktikum 6 – Traversal DFS

```
graph = {  
    "A": ["B", "C"],  
    "B": ["D", "E"],  
    "C": [],  
    "D": [],  
    "E": []  
}  
  
def dfs(graph, node, visited=None):  
    if visited is None:  
        visited = set()  
  
    if node not in visited:  
        print(node, end=" ")  
  
        visited.add(node)  
  
        for neighbor in graph[node]:  
            dfs(graph, neighbor, visited)
```

```
dfs(graph, "A")
```

Output: A B D E C

Praktikum 7 – Perbandingan BFS dan DFS

Bandingkan hasil:

BFS: A B C D E

DFS: A B D E C

Praktikum 8 – Representasi Adjacency Matrix

```
matrix = [  
    [0,1,1,0],  
    [1,0,0,1],  
    [1,0,0,1],  
    [0,1,1,0]  
]
```

```
for row in matrix:  
    print(row)
```

Praktikum 9 – Simulasi Jaringan Komputer

Representasikan:

Komputer A

Komputer B

Komputer C

Komputer D

ke dalam bentuk Graph.

Praktikum 10 – Simulasi Rute Kota

Buat Graph:

Bandung

Jakarta

Bogor

Cirebon

Kemudian lakukan traversal BFS dan DFS.

7. STUDI KASUS

Studi Kasus 1

Jaringan pertemanan media sosial.

Studi Kasus 2

Jaringan komputer kampus.

Studi Kasus 3

Peta rute perjalanan.

Studi Kasus 4

Hubungan antar halaman website.

Studi Kasus 5

Jaringan distribusi logistik.

Studi Kasus 6

Analisis hubungan pelanggan dalam bisnis.

8. TUGAS PRAKTIKUM

Tugas 1

Buat Graph dengan 5 node.

Tugas 2

Representasikan Graph menggunakan Adjacency List.

Tugas 3

Representasikan Graph menggunakan Adjacency Matrix.

Tugas 4

Implementasikan BFS.

Tugas 5

Implementasikan DFS.

Tugas 6

Bandingkan hasil BFS dan DFS.

Tugas 7

Jelaskan perbedaan Queue dan Stack pada BFS dan DFS.

Tugas 8

Buat Graph jaringan pertemanan sederhana.

Tugas 9

Buat Graph rute antar kota.

Tugas 10

Analisis kelebihan dan kekurangan BFS dan DFS.

9. DISKUSI

1. Apa yang dimaksud Graph?
2. Apa perbedaan Node dan Edge?
3. Apa perbedaan Graph dan Tree?
4. Apa fungsi Adjacency List?
5. Apa fungsi Adjacency Matrix?
6. Mengapa BFS menggunakan Queue?
7. Mengapa DFS menggunakan Stack?
8. Kapan BFS lebih cocok digunakan?
9. Kapan DFS lebih cocok digunakan?
10. Apa manfaat Graph dalam dunia nyata?

10. FORMAT LAPORAN

1. Judul Praktikum
2. Tujuan
3. Dasar Teori
4. Langkah Praktikum
5. Hasil dan Screenshot
6. Analisis
7. Kesimpulan